SQL JOIN ASSIGNMENT

Question 1: Retrieve all customers who have placed at least one order.

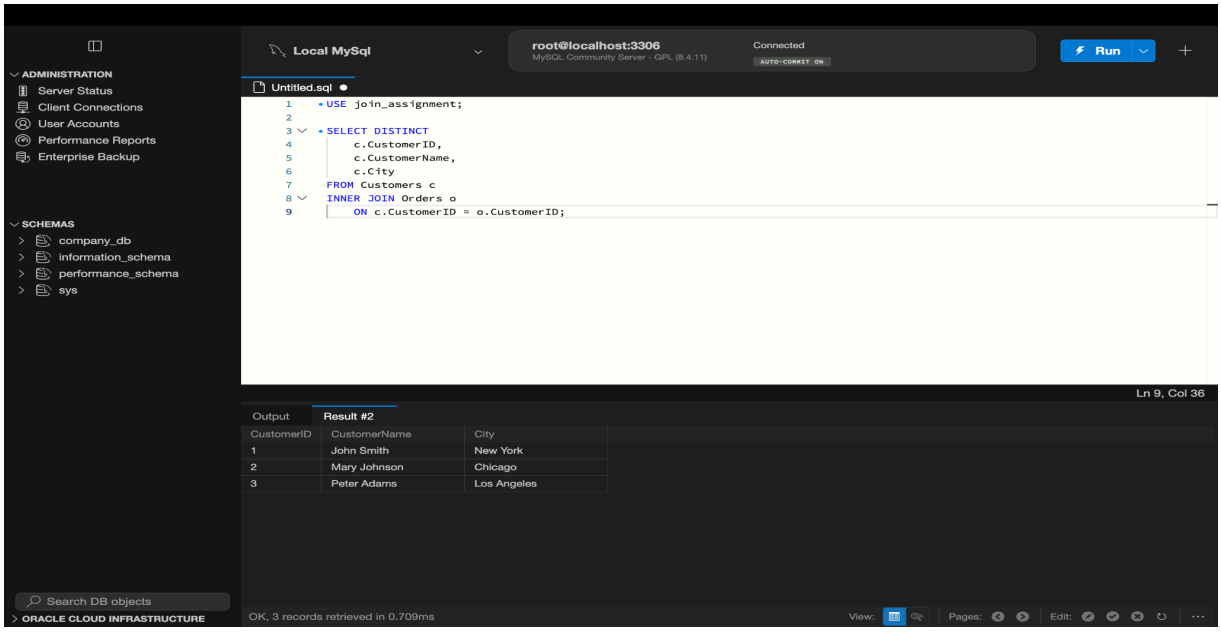
SQL Query:

```
USE join_assignment;

SELECT DISTINCT
    c.CustomerID,
    c.CustomerName,
    c.City
FROM Customers c
INNER JOIN Orders o
    ON c.CustomerID = o.CustomerID;
```

Explanation:
The INNER JOIN retrieves only those customers who have placed at least one order. The DISTINCT keyword prevents duplicate customer records.

Practical Proof:



Question 2: Retrieve all customers and their orders, including customers who have not placed any orders.

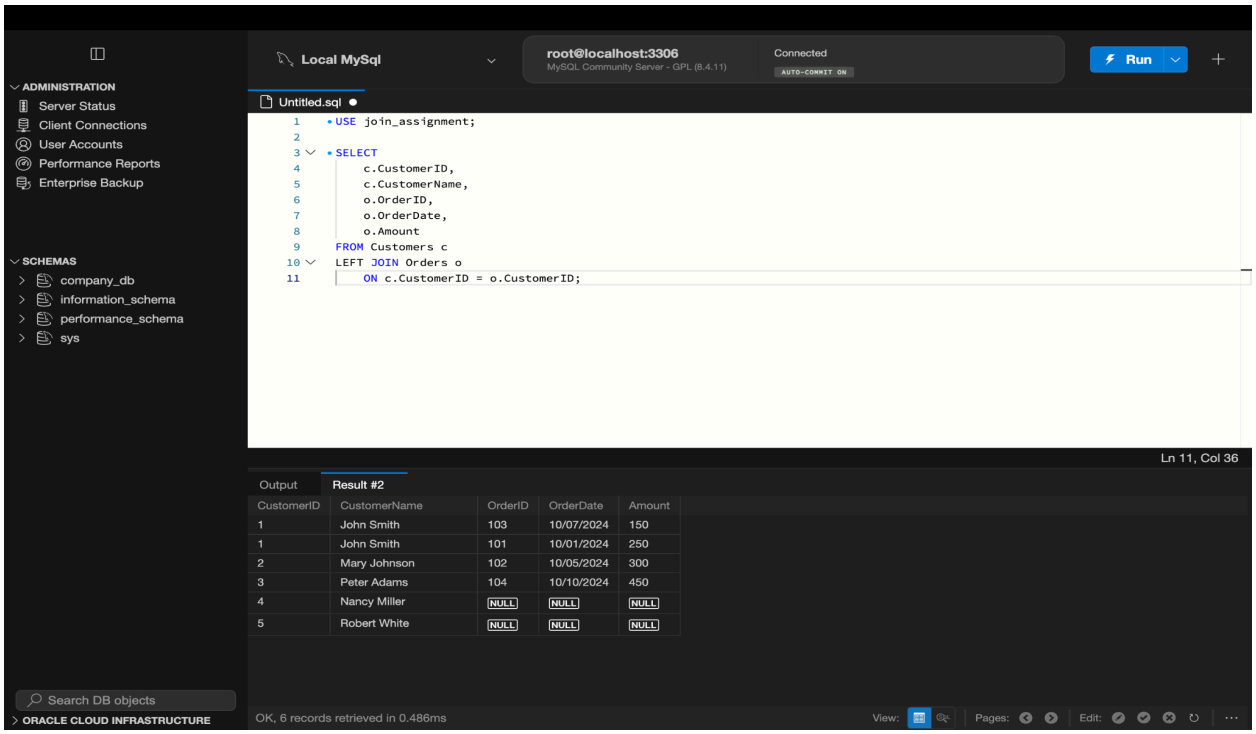
SQL Query:

```
USE join_assignment;

SELECT
    c.CustomerID,
    c.CustomerName,
    o.OrderID,
    o.OrderDate,
    o.Amount
FROM Customers c
LEFT JOIN Orders o
    ON c.CustomerID = o.CustomerID;
```

Explanation:
The LEFT JOIN retrieves all customers, including those who have not placed any orders. For customers without orders, the order-related columns display NULL.

Practical Proof:



Question 3: Retrieve all orders and their corresponding customers, including orders placed by unknown customers.

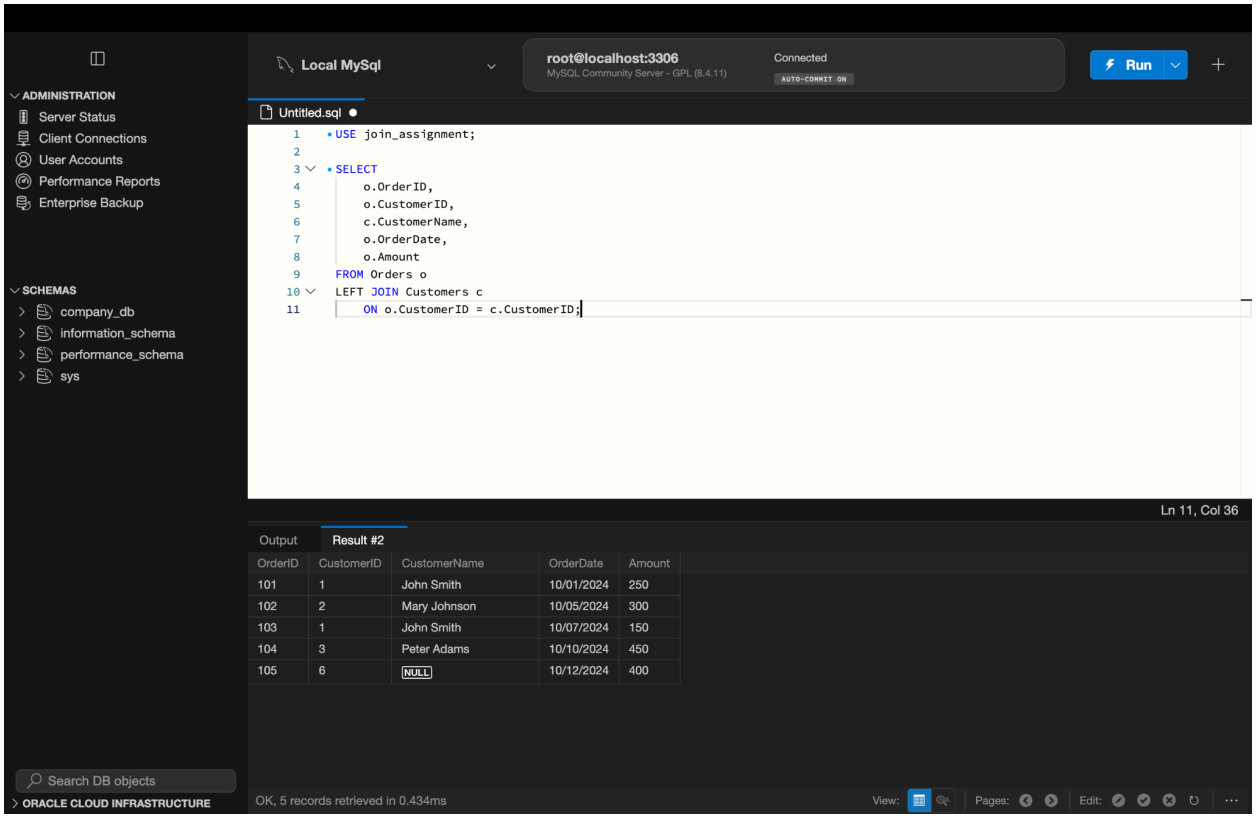
SQL Query:

```
USE join_assignment;


SELECT
    o.OrderID,
    o.CustomerID,
    c.CustomerName,
    o.OrderDate,
    o.Amount
FROM Orders o
LEFT JOIN Customers c
    ON o.CustomerID = c.CustomerID;
```

Explanation:
The LEFT JOIN retrieves all orders, including orders that do not have a matching customer in the Customers table. In this example, OrderID 105 belongs to CustomerID 6, who does not exist in the Customers table. Therefore, the CustomerName column displays NULL for that order.

Practical Proof :



Question 4: Display all customers and orders, whether matched or not (FULL OUTER JOIN).

SQL Query:

MySQL does not directly support `FULL OUTER JOIN`, so we use `LEFT JOIN`, `RIGHT JOIN`, and `UNION` to achieve the same result.

USE join_assignment;

SELECT

c.CustomerID,

c.CustomerName,

o.OrderID,

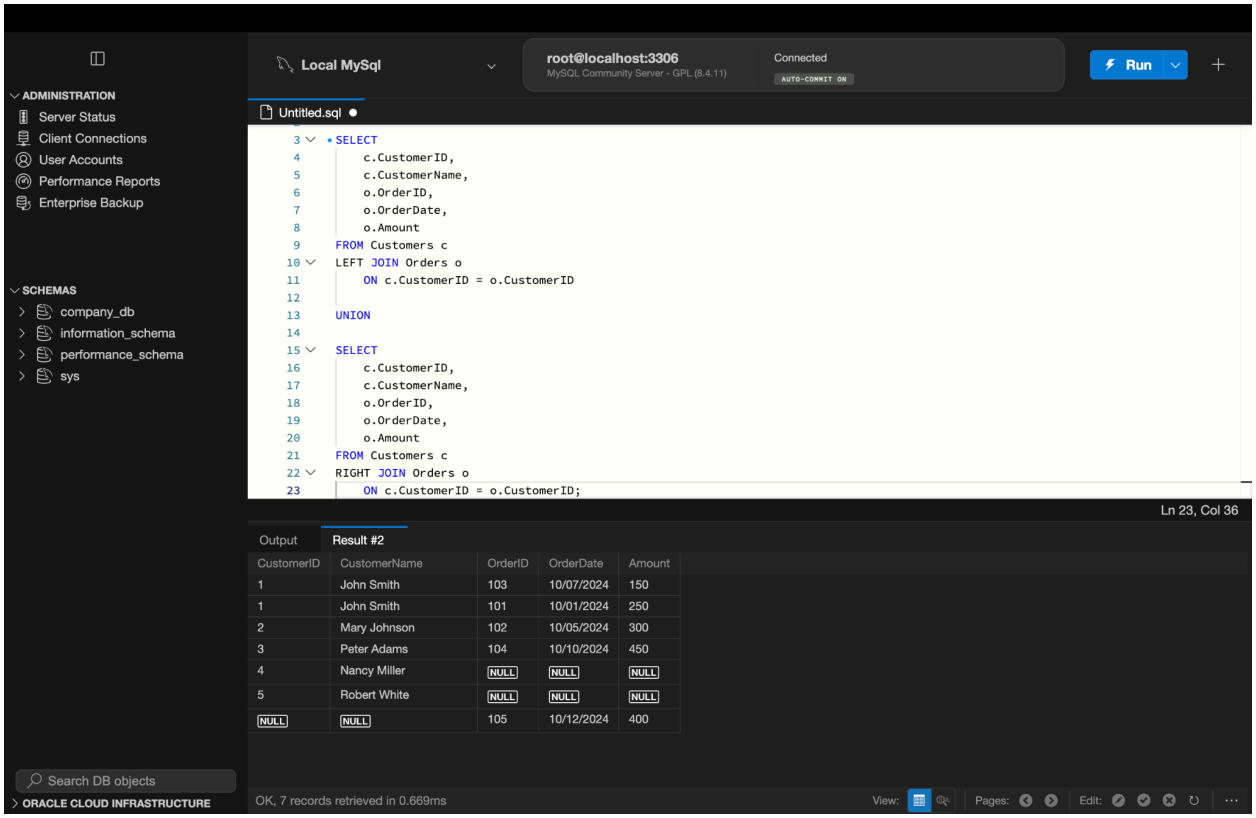
```
    o.OrderDate,
    o.Amount
FROM Customers c
LEFT JOIN Orders o
    ON c.CustomerID = o.CustomerID

UNION

SELECT
    c.CustomerID,
    c.CustomerName,
    o.OrderID,
    o.OrderDate,
    o.Amount
FROM Customers c
RIGHT JOIN Orders o
    ON c.CustomerID = o.CustomerID;
```

Explanation:
A FULL OUTER JOIN returns all matching and non-matching records from both tables. The LEFT JOIN includes all customers, while the RIGHT JOIN includes all orders. The UNION combines the results and removes duplicate rows.

Practical Proof:



Question 5: Find customers who have not placed any orders.

SQL Query:

USE join_assignment;

SELECT

c.CustomerID,

c.CustomerName,

c.City

FROM Customers c

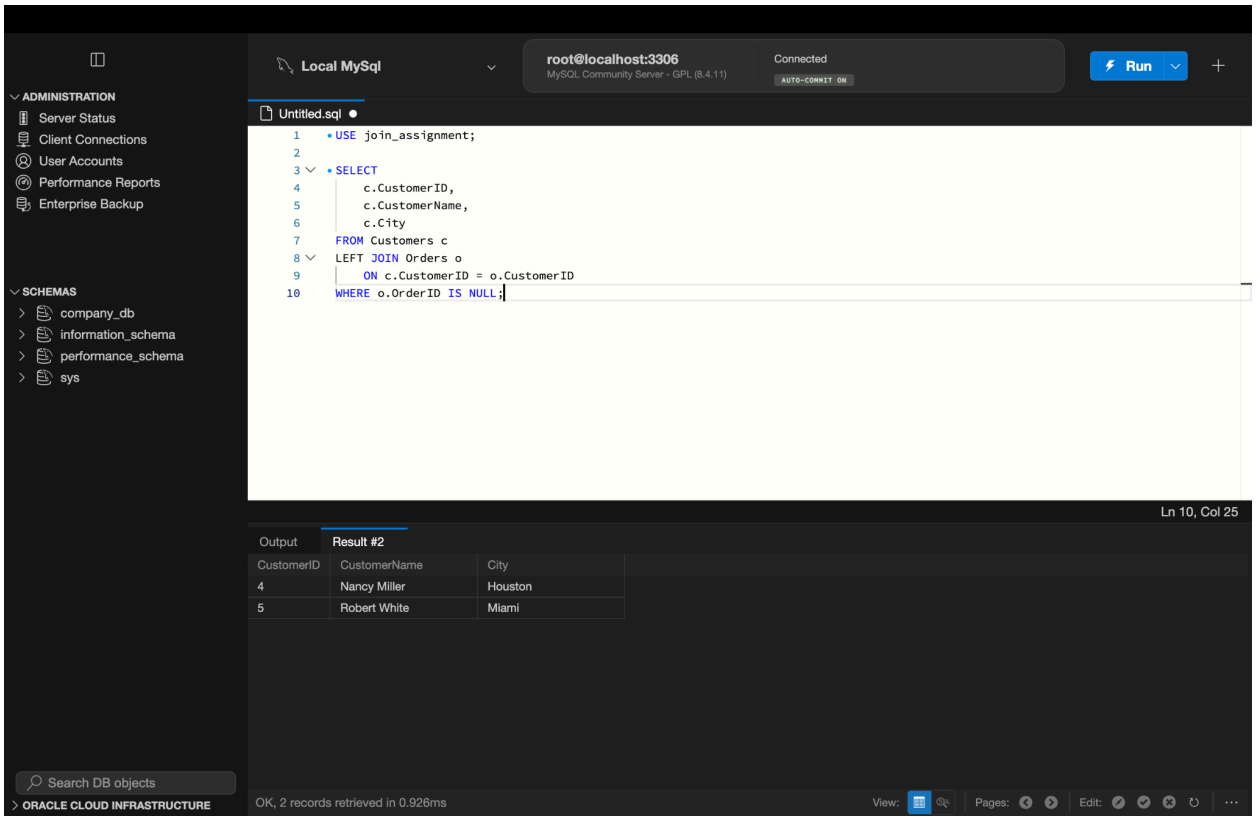
LEFT JOIN Orders o

ON c.CustomerID = o.CustomerID

WHERE o.OrderID IS NULL;

Explanation:
The LEFT JOIN retrieves all customers, including those without orders. The condition `WHERE o.OrderID IS NULL` filters the results to display only customers who have not placed any orders.

Practical Proof:



Question 6: Retrieve customers who made payments but did not place any orders.

SQL Query:

USE join_assignment;

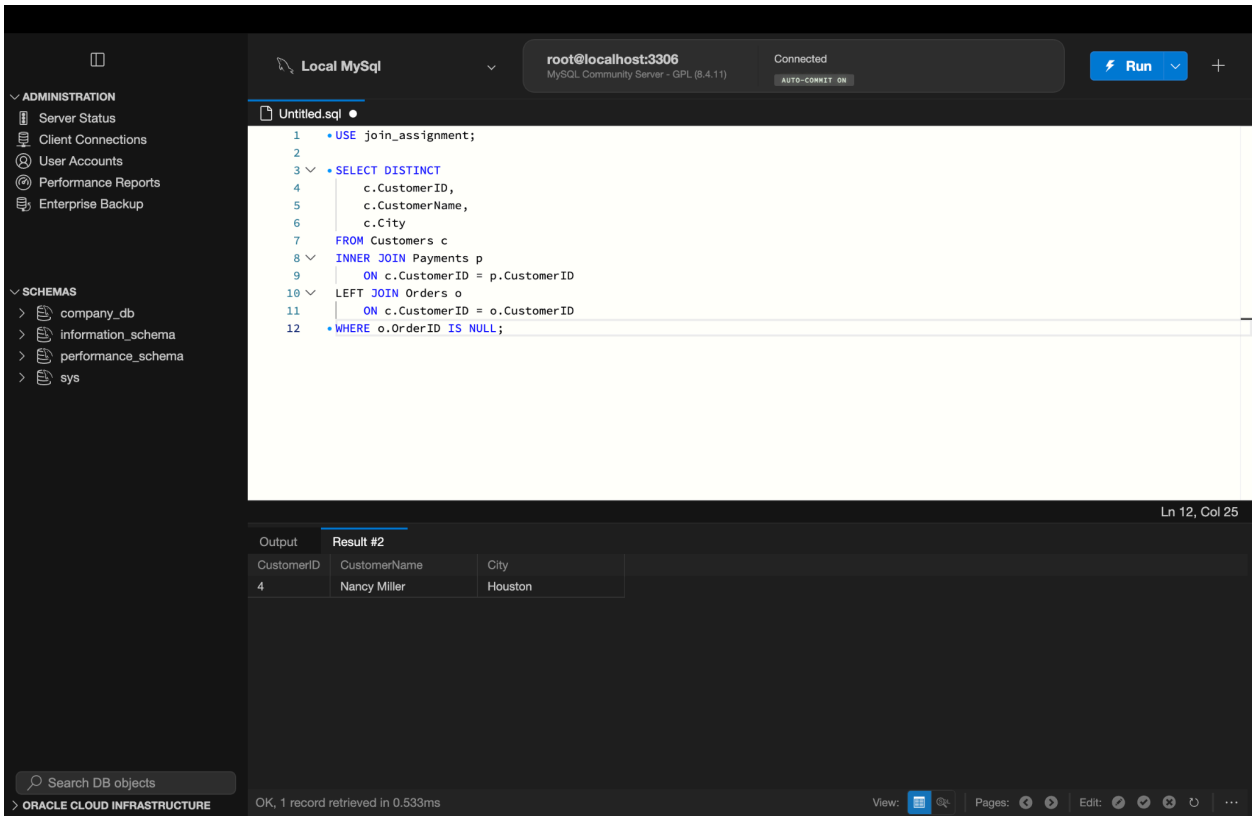
SELECT DISTINCT

c.CustomerID,

```
    c.CustomerName,
    c.City
FROM Customers c
INNER JOIN Payments p
    ON c.CustomerID = p.CustomerID
LEFT JOIN Orders o
    ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

Explanation:
The INNER JOIN identifies customers who have made payments. The LEFT JOIN checks whether those customers have placed any orders. The condition `WHERE o.OrderID IS NULL` displays only customers who made payments but did not place any orders.

Practical Proof:



Question 7: Generate all possible combinations between Customers and Orders.

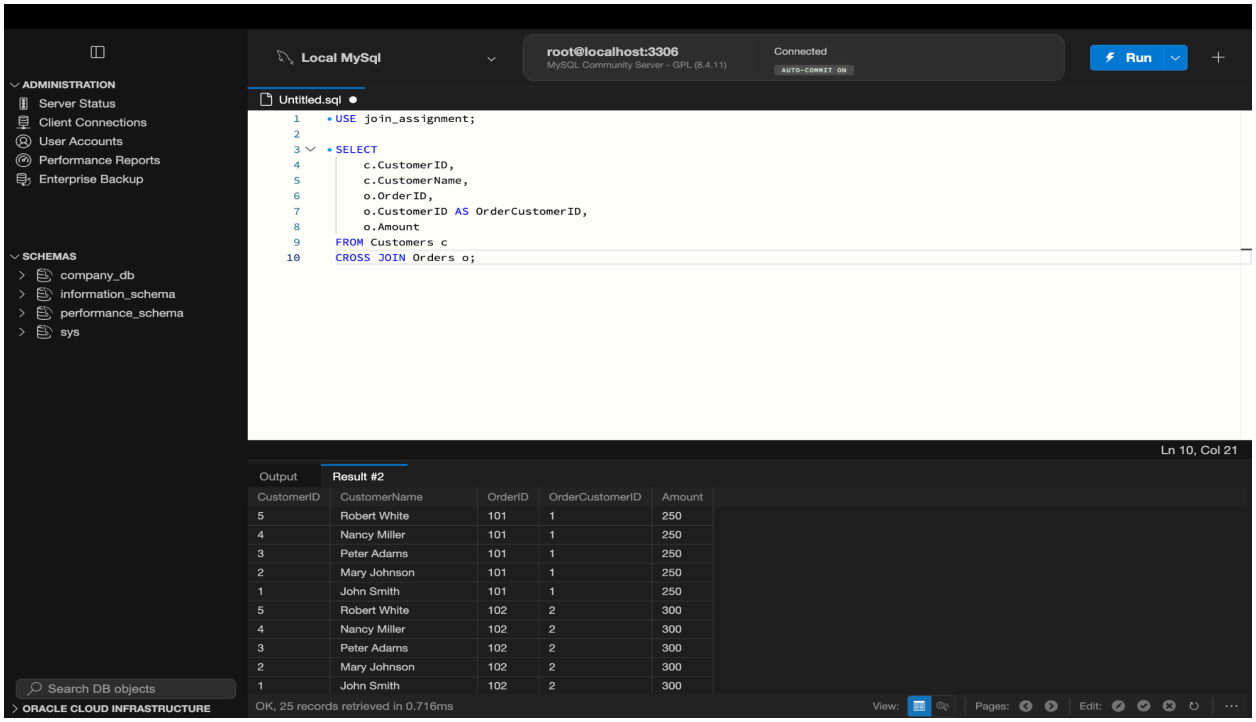
SQL Query:

```
USE join_assignment;


SELECT
    c.CustomerID,
    c.CustomerName,
    o.OrderID,
    o.CustomerID AS OrderCustomerID,
    o.Amount
FROM Customers c
CROSS JOIN Orders o;
```

Explanation:
The CROSS JOIN produces every possible combination of rows from the Customers and Orders tables. Since there are 5 customers and 5 orders, the total number of combinations is 5 × 5 = 25.

Practical Proof:



Question 8: Show all customers with order and payment amounts.

SQL Query:

USE join_assignment;

SELECT

c.CustomerID,

c.CustomerName,

o.OrderID,

o.Amount AS OrderAmount,

p.PaymentID,

p.Amount AS PaymentAmount

FROM Customers c

LEFT JOIN Orders o

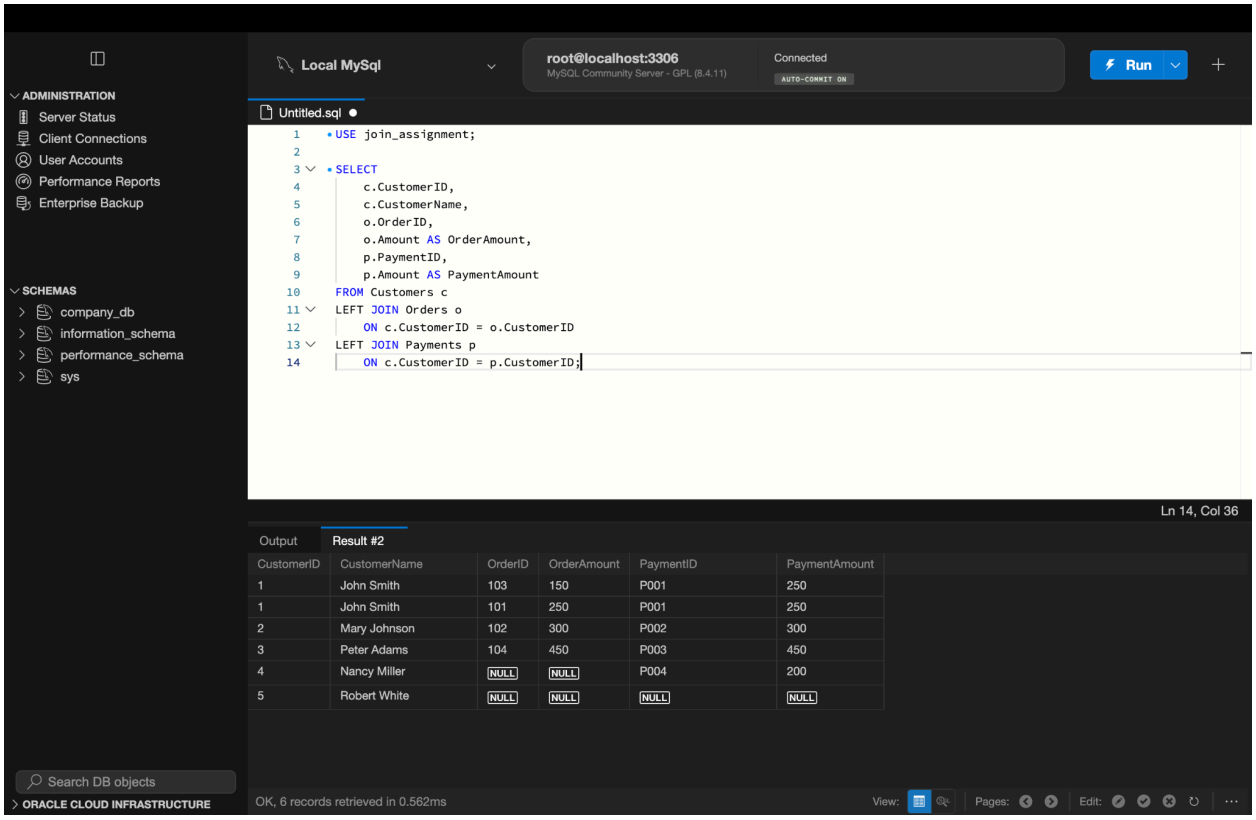
ON c.CustomerID = o.CustomerID

LEFT JOIN Payments p

ON c.CustomerID = p.CustomerID;

Explanation:
The LEFT JOIN combines the Customers, Orders, and Payments tables. It displays every customer along with their order and payment amounts, including customers who have no orders or payments. The NULL values indicate that the corresponding order or payment does not exist.

Practical Proof:



Question 9: Retrieve all customers who have both placed orders and made payments.

SQL Query:

```
USE join_assignment;


SELECT DISTINCT
    c.CustomerID,
    c.CustomerName,
    c.City
FROM Customers c
INNER JOIN Orders o
    ON c.CustomerID = o.CustomerID
INNER JOIN Payments p
    ON c.CustomerID = p.CustomerID;
```

Explanation:
The INNER JOIN combines the Customers, Orders, and Payments tables. It retrieves only customers who have both placed at least one order and made at least one payment. The DISTINCT keyword prevents duplicate customer records.

Practical Proof:

